This post was published to Intuitive Information at 11:21:03 PM 4/1/2013

Just another generalized version of Hamming Code – Hypercube representation

Ok, want to play a game? Let's say you are pretending to be a magician, and there're 7 audiences sitting in front of you.

1. Setting up:
    a. Number your audiences from 1 to 7.
    b. Ask ones numbered 1, 2, 3, and 4 to think about a binary number either 0 or 1 and then let the others know but not you.
    c. Then, the one with number 5 will sum all the binary numbers of those numbered 1, 2, and 4. If the result is even, he or she will choose 0; otherwise, 1.
    Similarly, the one with number 6 will come to the audiences 1, 2, and 3; and the one with number 7 will reach to the audiences 2, 3, and 4 to do the same thing.
    d. Ask them to discuss in group and choose no more than 1 person will cheat on me. That one will decide whether to flip his or her binary number or not.

    Remember: they do it secretly without letting you know what binary number they are holding.

2. Game gets started: Ask them to disclose the secret numbers they are holding. Then after thinking for seconds, you say: "You, the one with number 5, are cheating on me" if one of them actually flipped the binary number; otherwise, "well, thanks for no one cheating me". That's magic.

If you were interested in this 'magic' trick, read on. If you already knew what's behind the scene and wanted to know how I could approach to this trick with 2^n -1 audiences, read on. Otherwise, just turn off this browser tab.

Trick's explanation:

So, how could we figure out whether there's one of our audience is cheating or not? Generally speaking, the information of the other audiences is helping us. No matter they are numbered from 1-4 who could think of any binary number as they want or from 5-7 whose binary number depends on the former group 1-4, their binary number containing some information to correct the whole group when one of them goes wrong. That's teamwork talking.

Technically speaking, let's say the audiences from 1 to 4 came up with the bits: 1, 0, 1, and 1 respectively. Following the rule above:

- The audience 5 will have the value 0 as bit5 = (bit1+ bit2+ bit4) div 2 = (1+0+1) div 2 = 0
- Similarly, bit6 = (bit1 + bit2 + bit3) div 2 = (1 + 0 + 1) div 2 = 0
- And bit7 will be 0.

If the one chosen by the group to be a liar is 5, and he didn't do anything then the rule above would stay true. Otherwise, the bit5 would not satisfy the rule above. We couldn't say 5 is a liar right away because probably the liar could be 1, 2, or 4 (remember that just only one of them is a liar). However,

because the audience 6 and 7 satify the rule, the audiene 1, 2 or 4 is not the liar. For that reason, 5 is the liar. Confused yet? Try another explanation followed by the figure below:

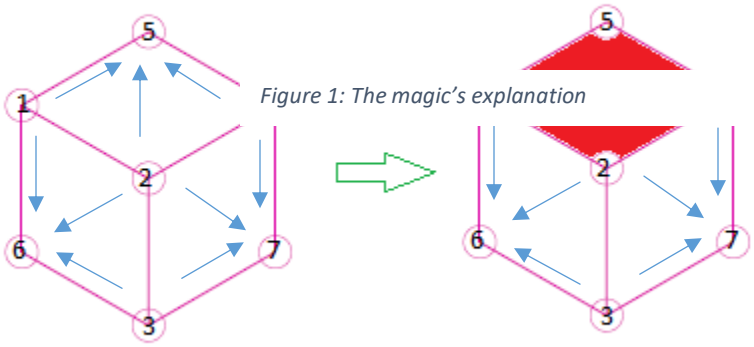


Figure 1: The magic's explanation

The audiences are divided into 3 groups arranged as the figure 1. We go through the rule step by step, and figure out the group of audiences 1, 2, 4 and 5 have a problem but the others are ok. As audience 5 is the only one doesn't belong to the other groups. So that one definitely is the liar.

Would it be still true if the liar is another one than audience 5? Yes, it would. Because of the arrangement among audiences which allows us to identify every audience based on relationships between him or her and the 3 groups. For instances, audience 2 is the only one staying in all 3 groups or audience 4 is the only one staying in groups with 5 and 7 but not with 6 (please refer to the *figure 1*).

The gist of the trick here is the way how we arranged our audiences in a special graph in which they could cross-check the others in their group.

Naysayers: Well, I could do the same trick without using the graph above. I will just group 3 audiences and then let one of them pick any binary number for the whole group. If one of us lied, you would know right away as he or she would differ from the others in their bits.

Me: Yep, you could do that. The difference between your trick and mine is just the amount of the redundant information that is added to the data source in order to make the group be able to self-correct when something wrong happen.

In my trick I added 3 bits to the source with 4 bits, so the info rate is 4/7; whereas, you added 2 bit to the source with only one bit at the rate 1:3.

So, not only the special arrangement of our audiences but also the amount of redundant information will be taken into account. That's it for the trick. If you want to know where the special graph comes from or whether there are other graph having that kind of structure in this world, please keep reading.

Hamming Code

Now, the funny part is over, but the exciting part just gets started. The trick I've just shown you is the visual representation of Hamming (7, 4). Hamming codes in general are codes with length 2^m-1 bits in which m is the number of parity bits included to enable the self-correcting ability of the bit blocks and 2^m-m-1 is the length of source bits.

Parity bits	Total bits	Data bits	Name	Rate
2	3	1	Hamming(3,1) (Triple repetition code)	1/3 ≈ 0.333
3	7	4	Hamming(7,4)	4/7 ≈ 0.571
4	15	11	Hamming(15,11)	11/15 ≈ 0.733
5	31	26	Hamming(31,26)	26/31 ≈ 0.839
...				
m	$2^m - 1$	$2^m - m - 1$	Hamming $(2^m - 1, 2^m - m - 1)$	$(2^m - m - 1)/(2^m - 1)$

Source: Wikipedia

In information theory, Hamming code is the classical linear error-correcting block code that will be able to correct one-bit errors (as you already experienced how they could correct themselves through the game. The code have several kinds of representation such as matrix, algebra, polynomial, Trellis, or Tanner [1]. But none of them is **intuitive** enough to let us explain how to encode and decode naturally. And of course "If you can't explain it simply, you don't understand it well enough." - *Albert Einstein*. That's why I bring this very classical information theory problem on again in this blog. I hope my explanation on Hamming Code would be simpler at some degrees so that it could be a ground for future

development later on.

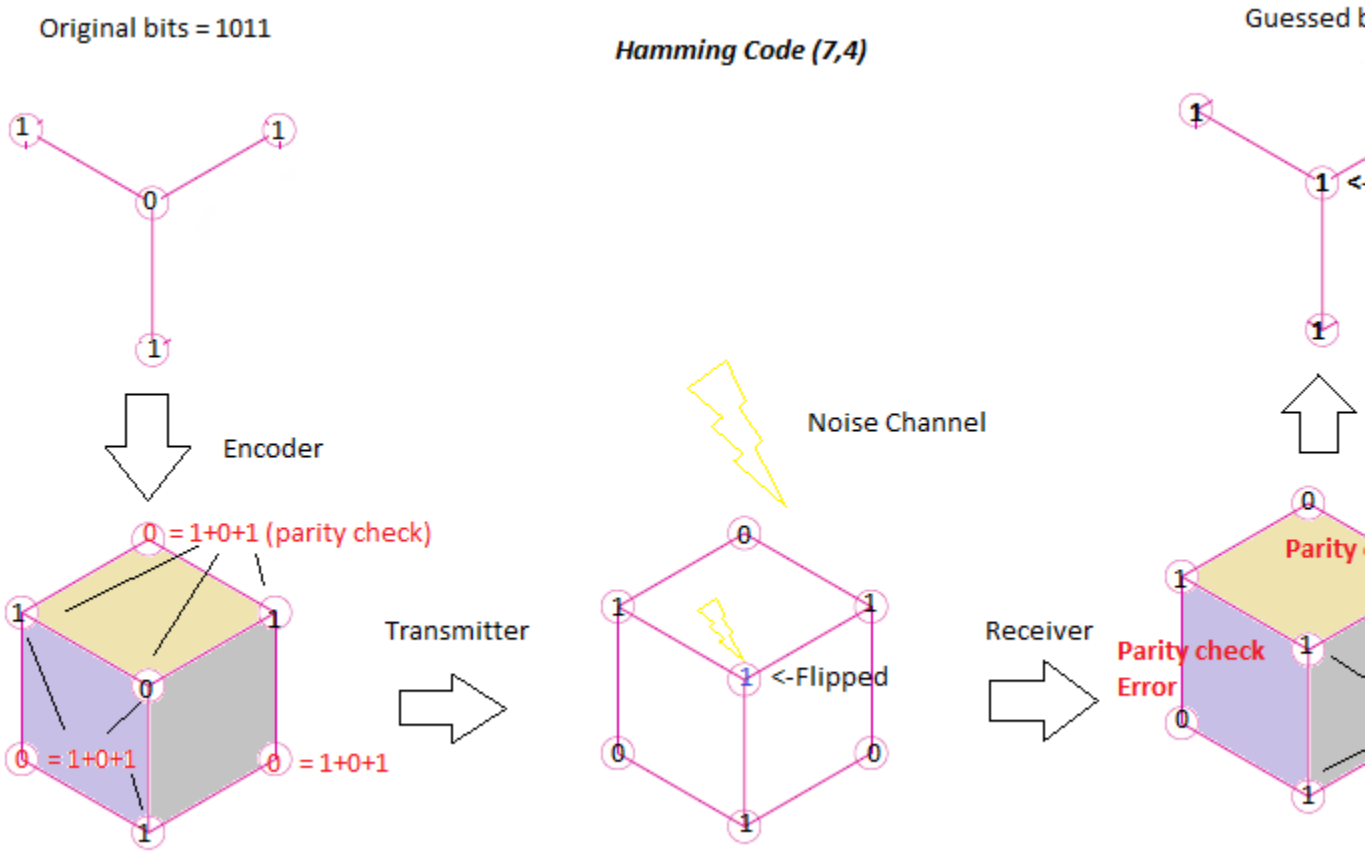


Figure 2: Hamming Code (7, 4) used in transmitting information

It's quite easy for you to understand the self-correcting mechanism in the figure 2 if you already mastered the aforementioned 'magic' trick, isn't it? However, if you are readers with sharp eyes, you will notice that the arrangement of bits looks like a cube that is missing a vertex and 3 line segments. Not just that, the bit length of a Hamming code is 2^m-1 with m parity bits. We could predict that a hypercube would be perfect to be used as such special bit arrangements.

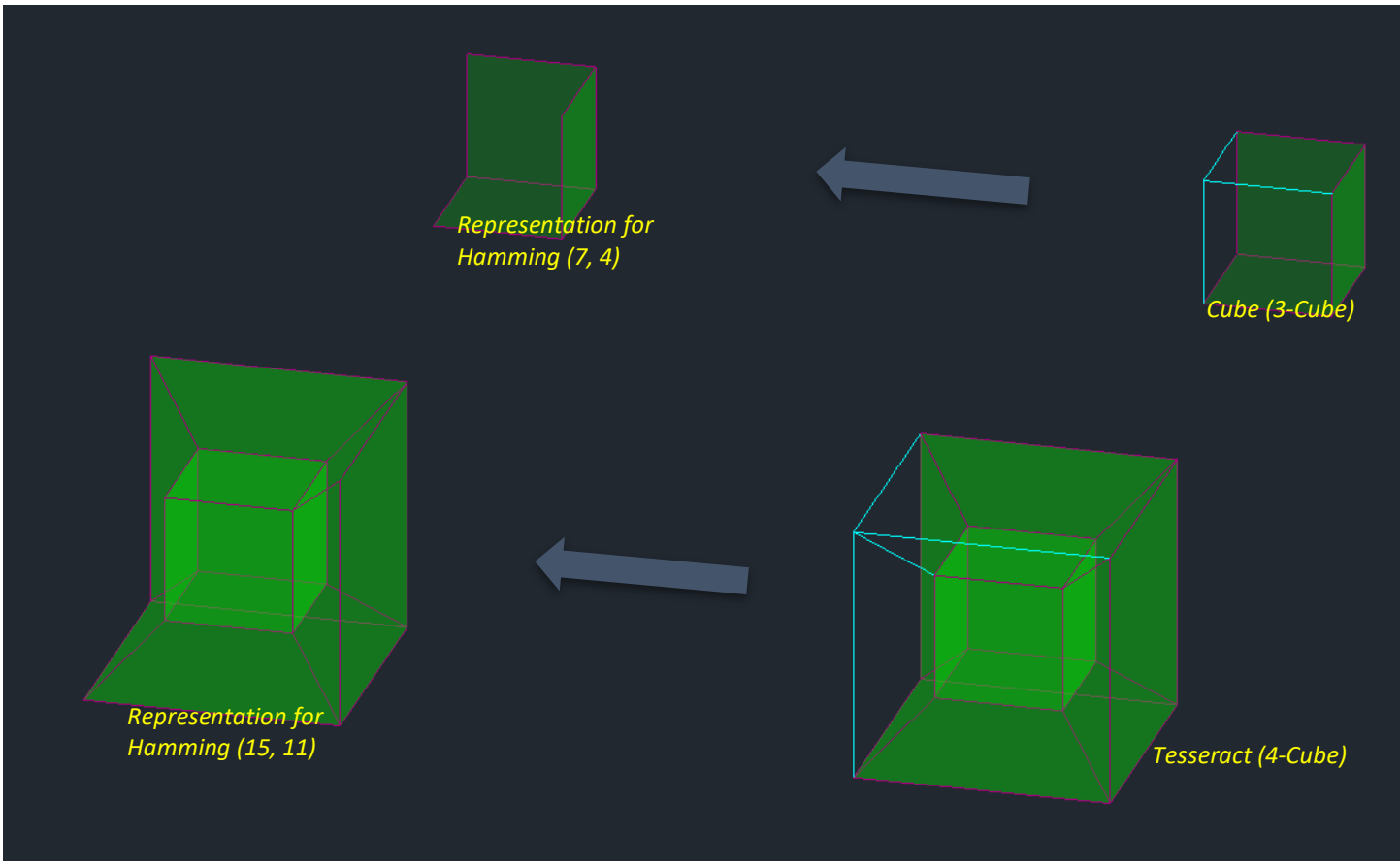
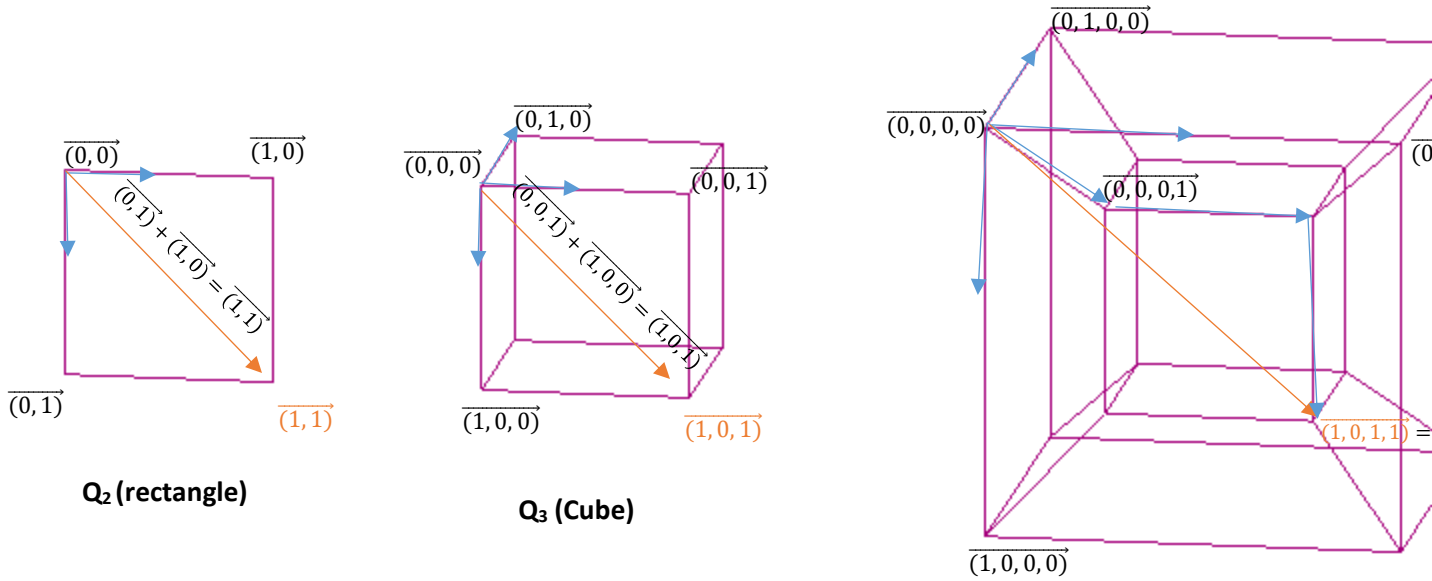


Figure 3: Hypercube vs. Hamming Graph Representation

Hypercube:

Generally speaking, *hypercube* is a cube in n-dimension space. To make it simple, we will use a *unit hypercube (shortly hypercube)* whose side has length one unit. We can get a detailed definition following this link in Wikipedia; however, I want to refer you to another definition that describes the most important characteristic of the hypercube: "*n-Cubes an n-dimensional hypercube, or n-cube, denoted by Q_n, is a graph that has vertices representing the 2n bit strings of length n. Two vertices are adjacent if and only if the bit strings that they represent differ in exactly one bit position.*"[2]



Q_2 (rectangle) **Q_3 (Cube)** **Q_4 (Tesseract)**

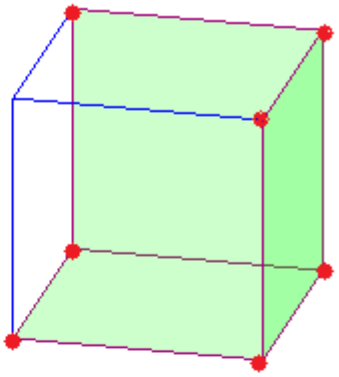
Figure 4: n-Cube Qn with n=2, 3, and 4.

To build an n-cube, we need to define its n unit vectors composed of n+1 bit strings of length n first and then using vector addition operations to position the 2^n -n-1 other bit strings. It sounds familiar to the definition of the Hamming codes, doesn't it?
That's enough for the introduction of Hypercube and Hamming code.

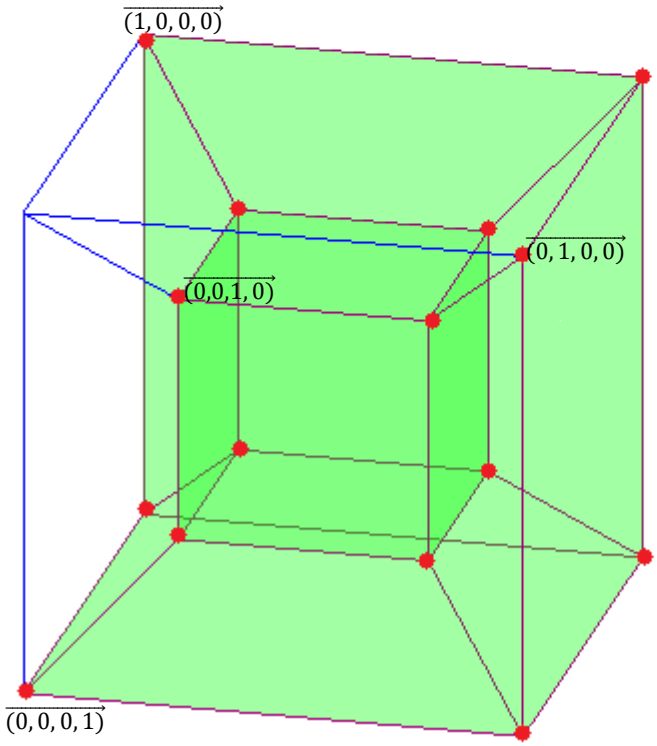
How to use Hypercube for building up Hamming Codes:

After having been reading so far, some of you would probably figure out the relationship between the Hamming Codes and n-Cubes. Let me go straight to the point. To make up an encoder and a decoder for a Hamming Code with length 2^n -1 bits, we need to find a special graph composed of n groups each of which contains 1 parity check bit. The relative position of each bit (vertex) to all groups is distinctive.

An n-Cube after removing one vertex and its n crossing edges is such a special graph. The explanation of the magic trick I have shown you before could prove this proposition for the 3-Cube instance. Let's do it with the Q_4 (tesseract).



Q_3 (Cube)



Q_4 (Tesseract)

The Q4 is which has 4 groups

References:

1. sd

2. Rosen, K. H. (2003). *Discrete mathematics and its applications*. Boston: McGraw-Hill.